

第2章 Linux进程管理

2.0 进程与线程知识回顾

- 2.1 Linux进程描述
- 2.2 Linux进程状态与转换
- 2.3 Linux进程标识与组织
- 2.4 Linux进程的系统堆栈
- 2.5 Linux进程的链表结构
- 2.6 Linux进程的家族关系
- 2.7 Linux进程控制
- 2.8 Linux进程与线程实现

本章教学目标

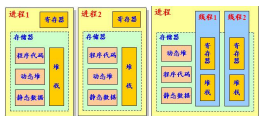
- 理解和掌握Linux的进程/线程描述机制
 - Linux进程描述符
 - Linux进程的系统堆栈结构
- 理解和掌握Linux进程的内存管理机制
 - 内核链表结构
 - Linux的进程家族结构
 - Linux进程系统启动过程
- 理解和掌握Linux进程创建与实现机制
 - fork(), vfork(), clone()的使用场景及实现机制的差别
 - do_fork()的工作机理
- 理解和掌握Linux线程创建与实现机制
 - Linux线程的实现机制
 - Linux用户线程内核线程的创建方法

2.0 进程与线程知识回顾

- Linux是多任务、多用户操作系统，一个任务(Task)就是一个进程(Process)。
- 每一个进程都具有一定的功能和权限，它们都运行在各自独立的虚拟地址空间。
- 在Linux中，进程是系统资源分配的基本单位，也是CPU的基本调度单位。
- 进程在OS内用PCB表示 (Process Control Block)，PCB是进程的属性之一。

2.0 进程与线程知识回顾

- 进程 (process)
 - 程序的一次执行及其所包含资源的总称
 - 程序：可执行程序代码
 - 资源：打开文件、挂起信号、地址空间、数据段等
- 线程 (thread)
 - 进程中活动的对象
 - 有独立的程序计数器、进程栈及一组进程寄存器



2.0 进程与线程知识回顾

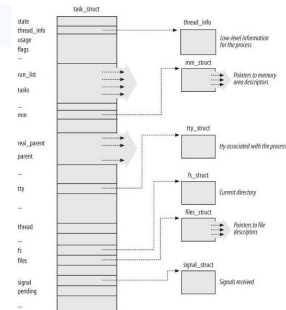
- 从形态角度
 - 一个进程可包含一个或多个线程
- 从调度角度
 - 进程是资源分配的基本单位
 - 线程是处理器调度的独立单位
- 从虚拟化角度
 - 进程提供两种虚拟机制
 - 虚拟处理器：进程独享处理器的假象
 - 虚拟内存：进程拥有系统内所有内存资源的假象
 - 线程之间可共享虚拟内存，但各自拥有独立虚拟处理器

2.0 进程与线程知识回顾

- 正文段
 - 存放进程运行的代码，描述进程需完成的功能
- 进程数据段
 - 存放正文段在执行期间所需的数据和工作区
 - 包括全局变量、动态分配的空间 (调用malloc函数)
 - 用户栈也在该数据段开辟，存放函数调用时的栈帧、局部变量等
- 系统堆栈
 - 每个进程捆绑一个，进程在内核态下工作时使用
 - 保存中断现场、执行函数调用时的参数和返回地址等
 - 其中最重要的数据结构是进程控制块

2.1 Linux进程描述

- 内核必须对每个进程进行清晰的描述。Linux内核利用数据结构 `task_struct` 来描述，它代表一个进程的PCB。
- `task_struct` 定义于 `include/linux/sched.h` 中，内核以此感知进程的存在。
- 当建立新进程的时候，Linux 为新的进程分配一个 `task_struct` 结构。



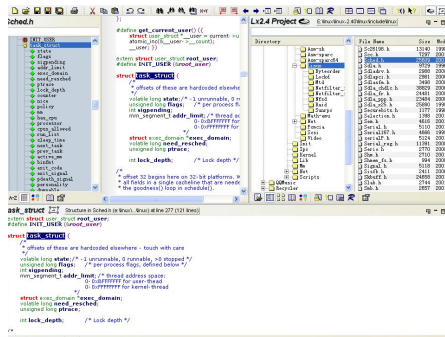
2.1 Linux进程描述符

- 进程描述符提供了内核所需了解的进程信息
- 进程描述符
 - 数据结构: `task_struct` 定义位置: `include/linux/sched.h`
 - `include/linux/sched.h` 中的 `struct task_struct`
- 进程描述符向量结构
 - 数据结构: `task[NR_TASKS]`
 - 定义位置: `include/linux/sched.h`
 - 定义格式
 - `struct task_struct *task[NR_TASKS] = {&init_task}`
 - `#define NR_TASKS 512`
- 说明
 - 全局变量 `NR_TASKS` 记录系统可容纳进程数，默认大小是12

2.1 Linux进程描述符

- 进程状态信息(state, flags, ptrace)
- 调度信息(static_prio, normal_prio, run_list, array, policy)
- 内存管理(mm, active_mm)
- 进程状态位信息(bimfmt, exit_state, exit_code, exit_signal)
- 身份信息(pid, tgid, uid, suid, fsuid, gid, egid, fsuid, fsgid)
- 家族信息(real_parent, parent, children, sibling)
- 进程耗间信息(realtime, utime, stime, starttime)
- 时钟信息(it_prof_expires, it_virt_expires, it_sched_expires)
- 文件系统信息(link_count, fs, files)
- IPC信息(sysvsem, signal, sighand, blocked, sigmask, pending)

Linux进程描述符的信息组成



2.1 Linux进程描述符

再看Linux 2.4内核:

```
struct task_struct {
    ...
    struct list_head run_list;
    struct task_struct *next_task, *prev_task;
    struct task_struct *p_opptr, *p_pptr, *p_cptr, *p_ysptr,
    *p_osptr;
    ...
    struct task_struct *pidhash_next;
    struct task_struct *pidhash_pprev;
    ...
    1) p_opptr指向进程的原始祖先, 通常和_pptr类似
    2) p_pptr指向进程的当前祖先
    3) p_cptr指向进程的最后一个(最近)子孙
    4) p_ysptr指向进程的下一个最早有一个最近兄弟
    5) p_osptr指向进程的下一个最古老有一个最近兄弟
}
```

2.2 Linux进程的状态及转换

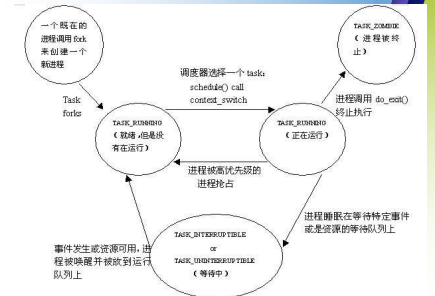
- task_struct中的state表示进程当前的状态:
volatile long state
- 功能: 表征进程的可行性
- 状态定义[include/linux/sched.h]


```
00105: #define TASK_RUNNING 0
00106: #define TASK_INTERRUPTIBLE 1
00107: #define TASK_UNINTERRUPTIBLE 2
00108: #define TASK_STOPPED 4
00109: #define TASK_TRACED 5
00110: #define TASK_ZOMBIE 6
00111: #define TASK_DEAD 7
```
- Linux 2.4与Linux 2.6的状态定义变化
 - 新增两种状态: TASK_TRACED, TASK_DEAD
 - 宏定义数值有较大差别

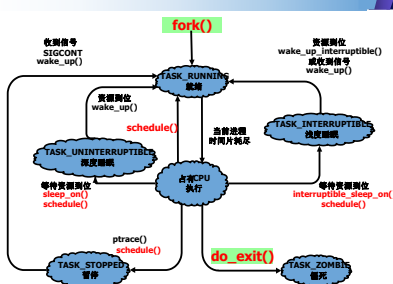
2.2 Linux进程的状态及转换

- 运行态就绪态
 - TASK_RUNNING 正在运行或已处于就绪只等待PU调度, 由current指针指向当前运行的进程。
 - TASK_TRACED 供调试使用
- 被挂起状态
 - TASK_INTERRUPTIBLE 可被信号或中断唤醒进入就绪队列
 - TASK_UNINTERRUPTIBLE 等待资源不可被其他进程中断
 - TASK_STOPPED 被调试暂停, 或收到SIGSTOP等信号
- 不可运行态
 - TASK_ZOMBIE 正在终止(已结束, 已释放内存、文件等资源, 但其父进程未接到通知, 描述符未释放), 等待父进程通过wait4()或waitpid()回收
 - TASK_DEAD 已退出且不需父进程回收的进程的状态

2.2 Linux进程的状态及转换



2.2 Linux进程的状态及转换



2.2 Linux进程的状态及转换

- ✓ sleep_on():
TASK_RUNNING → TASK_UNINTERRUPTIBLE
 - 运行进程申请资源无效时, 通过sleep_on()将进程从TASK_RUNNING切换到TASK_UNINTERRUPTIBLE状态, 加入等待队列中。
 - 一般引起状态变成TASK_UNINTERRUPTIBLE的资源申请, 都是对一些硬件资源的申请。如果得不到这些资源, 进程将不能执行下去, 不能由signal信号或时钟中断唤醒回到TASK_RUNNING状态。

2.2 Linux进程的状态及转换

- ✓ sleep_on_timeout():
TASK_RUNNING → TASK_UNINTERRUPTIBLE
- ✓ interruptible_sleep_on_timeout():
TASK_RUNNING → TASK_INTERRUPTIBLE
 - 虽然在申请资源或运行中出现了某种错误, 但是系统仍然给进程一次重新运行的机会。调用该函数将进程从TASK_RUNNING切换到TASK_INTERRUPTIBLE状态, 并等待规定的时间片长度再重新试一次。

2.2 Linux进程的状态及转换

✓ wake_up()

TASK_UNINTERRUPTIBLE→TASK_RUNNING

TASK_INTERRUPTIBLE→TASK_RUNNING

- 处于TASK_INTERRUPTIBLE状态的进程可在资源有效时被wake_up()、wake_up_interruptible()或wake_up_process()唤醒
- 处于TASK_UNINTERRUPTIBLE状态的进程不能由signal信号或时钟中断唤醒，只能由wake_up()或wake_up_process()唤醒。

2.3 Linux进程的标识与组织

- 成员名: **pid**
- 功能
 - 内核通过pid标识每个进程
 - pid与进程描述符之间一一对应关系
- 数据类型说明
 - pid_t**实际上是一个int类型
 - 取值范围: 0 ~ 32767
 - 最大值修改: /proc/sys/kernel/pid_max
 - 生成新pid: **get_pid()**函数
 - +1: 循环
- 获取进程pid
 - ps命令: 访问/proc/pid
 - getpid()←sys_getpid()

进程标识

2.3 Linux进程的标识与组织

- 功能
 - 查看系统中正在运行的进程
- 语法
 - ps [-ef][-n name][--tty][--p pids][--u users][--groups]

用户ID	进程号	父进程号	进程占用CPU的百分比	进程开始的时间和日期
root	1	0	0.0	20:51:04
root	2	1	0.0	20:51:04
root	3	1	0.0	20:51:04
root	4	1	0.0	20:51:04
root	7	1	0.0	20:51:04
root	5	1	0.0	20:51:04
root	6	1	0.0	20:51:04

进程已占用的时间 运行的命令

2.3 Linux进程的标识与组织

- 成员名: **pid_t tgid**
- 功能
 - 标识进程是否属于同组，组ID是第一个组内线程（父进程）的ID
 - 线程组中的所有线程共享相同的ID
- 组ID赋值相关操作

```
p->tgid = p->pid;
if (clone_flags & CLONE_THREAD)
    p->tgid = current->tgid;
```
- 单线程进程: tgid和pid相等
- 多线程进程: 组内所有线程tgid都相等，且等于父进程pid

进程组标识

2.3 Linux进程的标识与组织

- 功能: 控制用户对系统资源的访问权限
- 用户标识uid及组标识gid
 - 通常是进程创建者的uid和gid
- 有效用户标识euid及有效组标识egid
 - 有时系统会赋予一般用户暂时拥有root的uid和gid(作为用户进程的euid和egid)，以便于进行运作。
- 备份用户标识suid及备份组标识sgid
 - 在使用系统调用改变uid和gid时，利用其保留真正uid和gid
- 文件系统标识fsuid及文件系统组标识fsgid
 - 检查对文件系统访问权限时使用，通常suid及egid相等
 - 在NSF文件系统中，NSF服务器需要作为一个特殊的进程访问文件，此时只修改客户进程的suid和fsgid，而不改变euid及egid，可避免受恶意攻击

用户相关的进程标识信息

2.3 Linux进程的标识与组织

- 获取用户相关的进程标识信息的方法: getXXX()

```
#include <sys/types.h>
int main(int argc, char **argv) {
    pid_t my_pid, parent_pid;
    uid_t my_uid, my_euid;
    gid_t my_gid, my_egid;
    my_pid = getpid();
    parent_pid = getppid();
    my_uid = getuid();
    my_euid = geteuid();
    my_gid = getgid();
    my_egid = getegid();

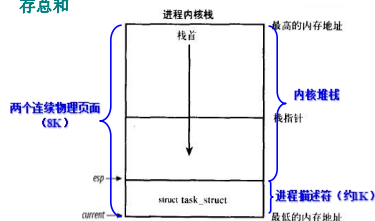
    printf("Process ID: %d\n", my_pid);
    printf("Parent ID: %d\n", parent_pid);
    printf("User ID: %d\n", my_uid);
    printf("Effective User ID: %d\n", my_euid);
    printf("Group ID: %d\n", my_gid);
    printf("Effective Group ID: %d\n", my_egid);
}
```

2.4 Linux进程的系统堆栈

- 每个进程都要单独分配一个系统堆栈。Linux的进程描述符与所属进程的内存堆栈放在一起。
- Linux为每个进程分配一个8KB大小的内存区域，用于存放该进程两个不同的数据结构:
 - 进程描述符task_struct 位于8K低端
 - 进程的内存堆栈 进程处于内核态时使用，位于8K高端
- 系统堆栈的结构特点
 - 8192 (2¹³) 字节，两个页框
 - 占用连续两个页框，且第一个页框起始地址为13的倍数

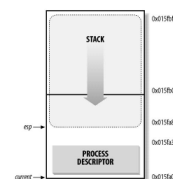
2.4 Linux进程的系统堆栈

- Linux系统进程个数限制
 - 所有进程的PCB及系统堆栈占用空间1/2的物理内存总和



2.4 Linux进程的系统堆栈

问题: 为什么这样存放?



这对当前进程的定位有好处
因为内核很容易从esp寄存器的值获得指向当前运行进程的PCB指针，由current宏完成。这样就既节省内存，管理上也比较方便。

2.4 Linux进程的系统堆栈

- C语言用如下的一个union结构来表示内核堆栈。

```
union task_union {
    struct task_struct task;
    unsigned long stack[INIT_TASK_SIZE/sizeof(long)];
};
```

2.4 Linux进程的系统堆栈

- 通常OS应有一个指向当前运行进程的CB指针，以便系统可以随时访问它。
- Linux却不是这样
 - 它利用其起始地址一定在堆栈所在的两页空间的低地址页边沿的特点，根据堆栈指针推算进程控制块的首地址。
- 从进程描述符和内核态堆栈之间的配对，内核可以很容易地从esp寄存器的值获得当前在CPU上运行的进程的控制块指针。

```
current宏:  
current=(struct  
task_struct(*)STACK_POINTER&0xFFFE000
```

2.4 Linux进程的系统堆栈

- 这个内存区是KB~2M字节大小，内核必须让sp有13位的有效位，以获得进程控制块的基本地址。这个工作由current宏来完成，该算法的伪代码如下。
- 通过将该值**低13位清零**，来得得到KB空间的起始地址，即PCB的地址。
- current宏可以看成当前进程的进程控制指针，在内核中直接使用，这样可以节省一个寄存器。
- 举例：current->pid返回正在执行的进程的PID

```
current宏:  
current=(struct  
task_struct(*)STACK_POINTER&0xFFFE000
```

2.4 Linux进程的系统堆栈

- 结构定义(/include/linux/sched.h)

```
union task_union {           #define INIT_TASK_SIZE 2048sizeof(long)
    struct task_struct task;
    unsigned long stack[INIT_TASK_SIZE/sizeof(long)];
};
```

- 进程描述符管理
 - 分配: `alloc_task_struct()`
 - 回收: `free_task_struct()`
 - 访问: `get_task_struct()`

2.4 Linux进程的系统堆栈

- ◆由current来获取当前进程的进程描述符

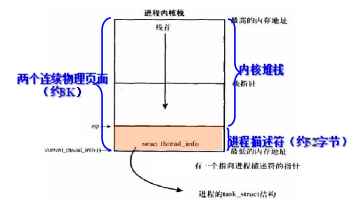
```
static inline struct task_struct * get_current(void)
{
    struct task_struct *current;
    __asm__ ("andi %%esp,%0;":"=r" (current): "0" (~0x191UL));
    return current;
}

#define current get_current()
```

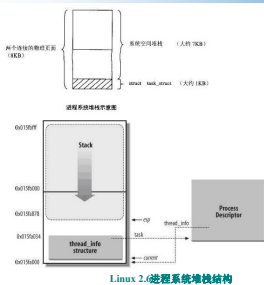
- 说明
- esp寄存器是CPU栈指针，%esp指向内核堆栈，存放栈顶单元地址
 - -8191UL表示最低13位为0，其余位全为1
 - 屏蔽掉%esp的最低13位后，得到“两个连续的物理页面”的开头（基地址），从而得到指向task_struct的指针

2.4 Linux进程的系统堆栈

- **Linux 2.6进程系统堆栈结构**
 - 进程描述符由lab分配器动态生成
 - 栈底用新结构**struct thread_info**, 指向进程描述符



2.4 Linux进程的系统堆栈



2.4 Linux进程的系统堆栈

- Linux 2.6进程系统堆栈结构
- Linux 2.6版本的内核栈底放的是thread_info结构, 通过thread_info里的task指针可以找到进程的task_struct。定义内核堆栈时使用的联合体:

```
union thread_union {
    struct thread_info thread_info;
    unsigned long stack[THREAD_SIZE/sizeof(long)];
};
```

2.4 Linux进程的系统堆栈

- Linux 2.6 进程系统堆栈结构
- thread_info 结构定义 [include/asm-x86/thread_info_32.h]
 - thread_info (52 个字节)

```
struct thread_info {
    struct task_struct *task; /* main task structure */
    struct exec_domain *exec_domain; /* execution domain */
    unsigned long flags; /* low level flags */
    unsigned long status; /* thread-synchronous flags */
    ... ..
}
```

2.4 Linux进程的系统堆栈

Linux 2.6进程系统堆栈结构

thread_union定义|include/linux/sched.h|

```
union thread_union {
    struct thread_info thread_info;
    unsigned long stack[THREAD_SIZE*sizeof(long)];
};
```

THREAD_SIZE定义|include/asm-x86/thread_info_32.h|

```
#ifndef CONFIG_4KSTACKS
#define THREAD_SIZE (4096)
#else
#define THREAD_SIZE (8192)
#endif
```

2.4 Linux进程的系统堆栈

Linux 2.6进程系统堆栈结构

- 根据内核的配置，THREAD_SIZE既可以是4K字节(1个页面)也可以是8K字节(2个页面)。thread_info是52个字节长。
- thread_info在这个内存区的开始处，内核堆栈从末端向下增长。
- Linux2.6中，进程描述符不在堆栈中，而分别通过task与thread_info指针使thread_info与进程描述符相联。

2.4 Linux进程的系统堆栈

Linux 2.6进程系统堆栈结构

- 由current来获取当前进程的进程描述符
 - 根据THREAD_SIZE大小，分别屏蔽掉内核栈的12-bit LSB(4K)或13-bit LSB(8K)，从而获得内核栈的起始位置

```
#define current get_current()
static inline struct task_struct * get_current(void) {
    return current_thread_info()->task;
}
static inline struct thread_info *current_thread_info(void) {
    struct thread_info *ti;
    __asm__ ("andi-%esp,%0; %%eax = (%ti) : %%eax (%esp-1)");
    return ti;
}
```

2.4 Linux进程的系统堆栈

Linux 2.6进程系统堆栈结构小结

- 与内核堆栈存放在一起的不再是进程控制块，而是thread_info结构。
- 但是由于thread_info结构中有一个指向进程控制块的task指针，所以系统仍然使用current宏查找进程控制块。
- 原因分析
 - PCB中最频繁被引用的是thread_info。
 - 随着版本的变化，进程的CB中的内容越来越多，所需空间越来越大，使得留给内核堆栈的空间过小，所以将部分进程控制块移出这个空间，只在核心栈保留了访问频繁的thread_info。

2.5 Linux进程的链表结构

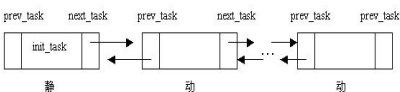
- 进程链表
- 运行队列链表
- 等待队列链表
- 哈希链表
- 指向当前进程的current指针

2.5 Linux进程的链表结构

- 为了对给定类型的进程，如所有在可运行状态下的进程进行有效搜索，内核维护了几个进程链表
 - 所有进程链表
 - 运行队列
 - 等待队列
 - pidhash表

2.5 Linux进程的链表结构

- 所有进程链表：**所有的进程用task_struct中的*prev_task和*next_task组成一个双向循环链表。链表的头和尾都是init_task。
- init_task的进程描述符是预先由编译器分配的，在运行的过程中保持不变。



在进程描述符中：struct task_struct *next_task, *prev_task;

2.5 Linux进程的链表结构

- 当CPU空闲时，内核调度程序寻找一个新的进程在PU上运行时，必须只考虑可运行进程，因为扫描整个进程链表效率很低。
- 由此，引入了TASK_RUNNING状态的进程双向循环链表，也叫**运行队列 (running queue)**。
- 进程描述符使用list_head用来实现运行队列

struct list_head run_list;

- list_head是Linux内核中定义的一个数据结构用来实现双向链表(include/list.h)

```
struct list_head {
    struct list_head *next, *prev;
};
```

2.5 Linux进程的链表结构

- 对可运行队列的一些操作函数

add_to_runqueue > 增加/删除一个可运行进程
del_from_runqueue > 增加/删除一个可运行进程
int nr_running; > 可运行队列的长度，可运行进程的个数
wake_up_process > 唤醒一个进程，将进程的状态设置为 TASK_RUNNING，使一个进程可运行
...

2.5 Linux进程的链表结构

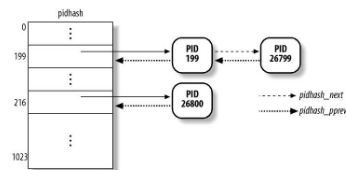
- 为了快速定位进程，系统以PID为关键字建立了一个哈希表（**pidhash表**）。若知道进程PID，可以通过该哈希表很快找到该进程。
- 例如：POSIX规范中规定的系统调用kill()是基于进程号向其它进程发送信号。因此，需要一种机制来高效地完成从进程号到进程描述符地址的转换。

```
#define PIDHASH_SZ (4096 >> 2)
struct task_struct *pidhash[PIDHASH_SZ];

#define pid_hashfn(x) (((x) >> 8) ^ (x)) & (PIDHASH_SZ - 1)
```

2.5 Linux进程的链表结构

- hash_pid(), unhashpid()在pidhash表中分别插入和删除一个进程。



2.5 Linux进程的链表结构

- 当要把除了TASK_RUNNING状态之外的进程组织在一起时，Linux使用了**等待队列**
 - TASK_STOPPED和TASK_ZOMBIE状态的进程不在专门的链表中，它们的父进程可以通过进程的PID或进程间亲属关系检索到子进程。
 - TASK_INTERRUPTIBLE和TASK_UNINTERRUPTIBLE状态的进程可以再分成很多类，每一类对应一个特定的事件。在这种情况下，进程状态提供的信息满足不了快速检索，因此，内核引进了另外的进程链表，叫做等待队列。

2.5 Linux进程的链表结构

- 等待队列使得进程可以在事件上进行的条件等待，并且当等待的条件为真时，由内核唤醒它们。尤其是对中断处理、进程同步和定时用处很大。
- 等待队列是以**双循环链表**为基础数据结构


```
struct __wait_queue {
    unsigned int flags;
    struct task_struct * task;
    struct list_head task_list;
};
typedef struct __wait_queue wait_queue_t;
```
- 其元素包括指向进程描述符的指针。每个等待队列都有一个等待队列头wait_queue_head_等待队列头是一个类型为wait_queue_head_的数据结构。

2.5 Linux进程的链表结构

- 要唤醒等待队列中**所有**睡眠的进程有时并不方便。
- 例如：如果两个或多个进程正在等待互斥访问某一要释放的资源，仅唤醒等待队列中一个进程才有意义。
 - 这个进程占有资源，而其他进程继续睡眠
 - 这就避免了所谓的“雷鸣般群羊”问题，即唤醒多个进程只为了竞争一个资源，而这个资源只能有一个进程访问，结果是其他进程必须再次回去睡眠。

2.5 Linux进程的链表结构

- 有两种处于阻塞状态的进程
 - 互斥进程等待队列元素的flag字段为1；由内核有选择地唤醒。
 - 非互斥进程flag值为0；总是由内核在事件发生时唤醒。
- 在等待队列上内核实现了一些操作函数
 - add_wait_queue()
 - remove_wait_queue()

2.5 Linux进程的链表结构

- 等待一个特定事件的进程能调用下面几个函数中的任一个
 - sleep_on()
 - sleep_on_timeout()
 - interruptible_sleep_on()
 - interruptible_sleep_on_timeout()
- 进程等待由需要等待的进程自己进行（调用）

2.5 Linux进程的链表结构

```
void sleep_on(wait_queue_head_t *q)
{
    unsigned long flags;
    wait_queue_t wait;
    wait.flags = 0;
    wait.task = current;
    current->state = TASK_UNINTERRUPTIBLE;
    add_wait_queue(q, &wait);
    schedule();
    remove_wait_queue(q, &wait);
}
```

2.5 Linux进程的链表结构

- 进程的唤醒**：利用wake_up()或者wake_up_interruptible()等一系列的宏，都让插入等待队列中的进程进入TASK_RUNNING状态

```
#define wake_up(x) __wake_up((x), TASK_UNI
#define wake_up_nr(x, nr) __wake_up((x), TASK_UNI
#define wake_up_all(x) __wake_up((x), TASK
#define wake_up_sync(x) __wake_up_sync((x)
#define wake_up_sync_nr(x, nr) __wake_up_syn
#define wake_up_interruptible(x) __wake_up((x), TASK
#define wake_up_interruptible_nr(x, nr) __wake_up
#define wake_up_interruptible_all(x) __wake_up((x)
#define wake_up_interruptible_sync(x) __wake_up_syn
#define wake_up_interruptible_sync_nr(x) __wake_up_
```

2.6 Linux进程的家族关系

- 所有进程都是**init进程**(pid=1)的后代
- 每个进程必须有一个父进程
- 每个进程可以拥有零个或者多个子进程
- 具有相同父进程的进程称为兄弟

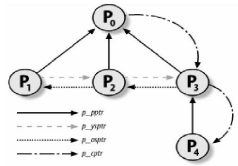
2.6 Linux进程的家族关系

- 进程之间具有父子关系，进程的父子关系是树型组织。
- 进程描述符中有几个域用来表示这样的关系

```
/*
 * pointers to (original) parent process, youngest child, younger sibling,
 * older sibling, respectively. (p->father can be replaced with
 * p->p_pptr->pid)
 */
struct task_struct *p_opptr, *p_pptr, *p_cptr, *p_ysptr, *p_osptr;
```

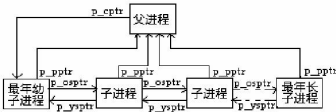
2.6 Linux进程的家族关系

- p_opptr**: original parent (1号进程或者创建它的父进程)
- p_pptr**: 父进程，有时候是调试时的调试监管进程
- p_cptr**: child (指向自己最年轻的子进程)
- p_ysptr**: 指向比自己年轻的兄弟进程
- p_osptr**: 指向比自己老的兄弟进程



2.6 Linux进程的家族关系

```
/* 父进程 (original parent) */
struct task_struct *p_opptr;
/* 父进程 (parent) */
struct task_struct *p_pptr;
/* 子进程 (youngest child) */
struct task_struct *p_cptr;
/* 新兄弟进程 (younger sibling) */
struct task_struct *p_ysptr;
/* 老兄弟进程 (older sibling) */
struct task_struct *p_osptr;
```



2.6 Linux进程的家族关系

- 父进程也有**生父**和**养父**之分。在task_struct结构中有个指针p_opptr指向其“original parent”也就是生父。另外还有个指针p_pptr则指向养父。
- 一个进程在创建之初其生父和养父是一致的，两个指针指向同一个父进程，但是在运行中p_pptr可以暂时地改变。
- 这种改变发生在一个进程通过系统调用trace()来跟踪另一个进程的时候，这时候被跟踪的进程的p_pptr的指针被设置成指向正在跟踪它的进程，那个进程就暂时成了被跟踪进程的养父。而被跟踪进程的p_opptr指向这个线程。
- 否则，就只好托付给系统中的init进程，所以这个init进程就好像是孤儿院。

2.6 Linux进程的家族关系

- Linux中的0号进程，是所有进程的祖先，在OS启动时创建。由它执行pu_idle()函数，当没有其他进程处于TASK_RUNNING的时候，调度程序会选择0号进程运行
- 0号进程创建1号进程，1号进程通常称为init进程。它首先创建一些后台进程来维护系统，然后进行系统配置，执行shell编写的初始化程序。然后转入用户态运行
- 思考题**: 查资料，了解linux2.6中的父子进程中间的**养父关系**。

2.6 Linux进程的家族关系

- 获取父进程
 - struct task_struct *my_parent=current->parent
- 获取所有子进程（实质是遍历list_head结构）

```
struct task_struct *task;
struct list_head *list;

list_for_each(list, &current->children) {
    task = list_entry(list, struct task_struct, sibling);
    /* task现在指向当前的某个子进程 */
}
```

2.7 Linux中的进程控制

- Linux进程的创建
- Linux进程的执行
- Linux进程的等待
- Linux进程的终止

2.7 Linux中的进程控制

- Linux的0号进程
 - 称为**0号进程**或**idle进程**或**swapper进程**
 - 维护Linux内核代码段、数据段及堆栈
 - 是唯一一个通过**静态分配**创建的进程arch/x86/kernel/init_task.c
 - INIT_MM, INIT_FS, INIT_FILES, INIT_SIGNALS, INIT_SIGHAND等宏初始化进程描述符的各个对应域
 - INIT_TASK完成init_task变量中进程描述符初始化
 - INIT_THREAD_INFO完成对init_thread_info中thread_info及内核堆栈的初始化
 - 主内核页全局目录存放swapper_pg_dir变量中
 - start_kerne函数**/init/main.c初始化内核所需所有数据结构、激活中断、创建**号内核线程**

2.7 Linux中的进程控制

- Linux的1号内核进程

- 又称为**init进程** (PID=1)
- 由0号进程在start_kernel()函数中调用rest_init创建

```
static void rest_init(void)
{
    kernel_thread(init, NULL, CLONE_FS | CLONE_FILES | CLONE_SIGNAL);
    unlock_kernel();
    current->need_resched = 1;
    cpu_idle();
}
```

- 调度程序选择到init进程时, init进程开始执行init()函数

2.7 Linux中的进程控制

- init()函数说明

- 实现从内核态到用户态的切换
- 为常规内核任务初始化一些必要的内核线程
 - kflushd: 刷新“脏”缓冲区中内容到磁盘, 归还内存
 - kswapd: 执行内存回收功能
- 调用系统调用execve()装入可执行程序init
 - init内核线程变成一个普通进程拥有自己的进程内核数据结构
 - 但init进程从不终止, 因为它创建和监控操作系统外层的所有进程的活动

2.7 Linux中的进程控制

- 传统的UNIX操作系统采用统一的方式来创建进程
 - 子进程复制父进程所拥有的资源
- 缺点:
 - 创建过程慢、效率低
 - 事实上, 子进程复制的很多资源是不会使用到的
- 引入三种不同的机制来解决这个问题
 - 写时复制技术
 - sys_fork()/sys_clone()/sys_vfork()
 - 轻量级进程允许父子进程共享许多数据结构阻塞父进程等

2.7 Linux中的进程控制

- 写时复制技术允许父子进程能读相同的物理页。
- 如果两者有一个进程试图写一个物理页, 内核就把这个页的内容拷贝到一个新的物理页, 并把这个新的物理页分配给正在写的进程
- fork()与以往不同, 实际开销只是复制父进程的页表以及给子进程创建唯一的PCB。
- 父进程的资源被设置为只读, 当父进程或子进程试图修改某些内容时, 内核才在修改前将该部分进行拷贝——写时复制

2.7 Linux中的进程控制

- 先通过复制页表项共享这个页面, 将父、子进程可写虚拟内存页的页表项均标志为只读
- 当父或子进程向该内存写入数据时, 就会引起一次页面异常缺页中断, 页面异常处理程序将重新分配一个物理页面, 并完成真正的内容复制
- fork()调用: 可理解为逻辑拷贝整个进程的地址空间, 仅当试图修改页面才真正的拷贝

2.7 Linux中的进程控制

- Linux提供了三个与进程创建相关的系统函数 sys_fork()/sys_clone()/sys_vfork()
 - 其中, sys_clone()创建轻量级进程, 必须指定要共享的资源
- exec()系统调用: 执行一个新程序
- exit()系统调用: 终止进程 (进程也可以因收到信号而终止)

2.7 Linux中的进程控制

- 引入三种机制优化进程创建效率

- 写时复制技术 (Copy-On-Writing)
- 轻量级进程
 - 允许父子进程共享页表、打开文件列表、信号处理等数据结构
 - 但每个进程应该有自己的程序计数器、寄存器集合、核心栈和用户栈
- vfork()
 - 新进程可共享父进程的内存地址空间

2.7 Linux中的进程控制

- 在终端输入命令, 由shell进程创建一个新进程
- 进程创建函数
 - pid_t fork(void);
 - pid_t vfork(void);
 - int clone(int (*fn)(void * arg), void * stack, int flags, void * arg);
 - 创建轻量级线程
- 三函数都调用同一内核函数do_fork() [/kernel/fork.c]

2.7 Linux中的进程控制

- 函数调用形式
 - do_fork(unsigned long clone_flag, unsigned long stack_start, struct pt_regs *regs, unsigned long stack_size, int *user *parent_tidptr, int *user *child_tidptr);
- 参数说明
 - clone_flag: 子进程创建相关标志
 - stack_start: 将用户态堆栈指针赋予子进程的sp
 - regs: 指向通用寄存器值的指针
 - stack_size: 未使用 (总设为0)
 - parent_tidptr: 父进程的用户态变量地址, 若需父进程与新轻量级进程有相同PID, 则需设置 CLONE_PARENT_SETTID
 - child_tidptr: 新轻量级进程的用户态变量地址, 若需让新进程具有同类进程的PID, 需设置 CLONE_CHILD_SETTID

2.7 Linux中的进程控制

参数标志	含 义
CLONE_FILES	父子进程共享打开的文件
CLONE_FS	父子进程共享文件系统信息
CLONE_IDLETASK	将PID设置为0 (只供idle进程使用)
CLONE_NEWNS	为子进程创建新的命名空间
CLONE_PARENT	新子进程与父进程拥有同一个父进程
CLONE_PTRACE	继续调试子进程
CLONE_SETTID	将TID写入用户空间
CLONE_SETTSL	为子进程创建新的TSL
CLONE_SIGHAND	父子进程共享信号处理函数
CLONE_SYSSPM	父子进程共享system V SEM、UNDO语义
CLONE_THREAD	父子进程放入相同的线程组
CLONE_VFORK	调用(vfork), 所以父进程准备睡眠等待子进程将其唤醒
CLONE_UNTRACED	防止跟踪进程在子进程上复制执行CLONE_PTRACE
CLONE_STOP	CLONE_STOPPED状态开始进程
CLONE_SETTSL	为子进程创建新的TSL (Shared-local storage)
CLONE_CHILD_CLEARTID	清除子进程的TID
CLONE_CHILD_SETTID	设置子进程的TID
CLONE_PARENT_SETTID	设置父进程的TID
CLONE_VM	父子进程共享地址空间

子进程创建CLONE参数标志说明

2.7 Linux中的进程控制

```
asmlinkage int sys_fork(struct pt_regs regs)
{
    return do_fork(SIGCHLD, regs.esp, &regs, 0);
}
```

- 说明
 - 子进程完全复制父进程的资源
 - 子进程的独立于父进程
 - 进程间数据共享需通过专门的通信机制来实现
- 返回值
 - 父进程执行fork()返回子进程的PID值
 - 子进程执行fork()返回0
 - 调用失败返回-1

2.7 Linux中的进程控制

```
asmlinkage int sys_vfork(struct pt_regs regs)
{
    return do_fork(CLONE_VFORK | CLONE_VM | SIGCHLD, regs.esp, &regs, 0);
}
```

- 说明
 - vfork()创建的子进程与父进程共享地址空间
 - 子进程作为父进程的一个单独线程在其地址空间运行
 - 子进程从父进程继承控制终端、信号标志位、可访问的主存区、环境变量和其他资源分配
 - 子进程对虚拟空间任何数据的修改都可对父进程所见
 - 父进程将被阻塞，直到子进程调用execve()或exit()
- 与fork()的关系
 - 功能相同，但fork()不拷贝父进程的页表项
 - 子进程只执行exec()时，vfork()为首选

2.7 Linux中的进程控制

- copy_process()将task_struct的vfork_done设为NULL
 - p->vfork_done = NULL;
- do_fork()将vfork_done指向一个特殊地址
 - if (clone_flags & CLONE_VFORK) {
p->vfork_done = &vfork;
init_completion(&vfork);
}
- 父进程阻塞直到子进程向其发送信号
 - if (clone_flags & CLONE_VFORK) {
freezer_do_not_count();
wait_for_completion(&vfork);
freezer_count();
if (unlikely (current->ptrace & PT_TRACE_VFORK_DONE)) {
current->ptrace_message = nr;
ptrace_notify ((PTTRACE_EVENT_VFORK_DONE << 8) | SIGTRAP);
}

vfork()调用的实现机制说明

2.7 Linux中的进程控制

- 调用mm_release()时，检查vfork_done是否为空
 - 若不为空，则向父进程发送信号

```
struct completion *vfork_done = tsk->vfork_done;
/* Get rid of any cached register state */
deactivate_mm(tsk, mm);
/* notify parent sleeping on vfork() */
if (vfork_done) {
    if (tsk->vfork_done == NULL)
        complete(vfork_done);
}
```

vfork()调用的实现机制说明

2.7 Linux中的进程控制

- 函数原型
 - int clone(int (*fn)(void *), void *child_stack, int clone_flag, void *arg);
- 参数说明
 - fn: 待执行的程序
 - child_stack: 进程所使用的堆栈
 - clone_flag: 由用户指定，可以是多个标志的组合
 - arg: 执行fn所需的参数
- 功能
 - 创建轻量级进程LWP的系统调用
 - 通过clone_flag控制

2.7 Linux中的进程控制

```
asmlinkage int sys_clone(struct pt_regs regs)
{
    unsigned long clone_flags;
    unsigned long newsp;
    int __user *parent_tidptr, *child_tidptr;

    clone_flags = regs.bx;
    newsp = regs.cx;
    parent_tidptr = (int __user *)regs.dx;
    child_tidptr = (int __user *)regs.di;
    if (!newsp)
        newsp = regs.sp;
    return do_fork(clone_flags, newsp, regs, 0,
        parent_tidptr, child_tidptr);
}
```

- fn及arg参数的获取
 - do_fork()返回时可通过进程切换机制 (mp+ret) 使CPU从stack中获取返回地址，并得到n地址

2.7 Linux中的进程控制

- CLONE_VM: 父子进程共享内存描述符及所有页表
- CLONE_FS: 父子进程共享文件系统信息
- CLONE_FILES: 父子进程共享文件描述符表
- CLONE_SIGHAND: 父子进程共享信号描述符
- CLONE_PTRACE: 若父进程被跟踪，子进程也被跟踪
- CLONE_PARENT: 父进程的real_parent登记为子进程的parent和real_parent
- CLONE_THREAD: 子进程加入父进程的线程组
- CLONE_STOPPED: 创建新进程但不运行之

clone()函数的常用CLONE标志

2.7 Linux中的进程控制

```
void* func(int arg) {
    .....
}

int main() {
    int clone_flag, arg;
    .....
    clone_flag = CLONE_VM | CLONE_SIGHAND | CLONE_FS | CLONE_FILES;
    stack = (char *)malloc(STACK_FRAME);
    stack += STACK_FRAME;
    retval = clone((void *)func, stack, clone_flag, arg);
    .....
}
```

2.7 Linux中的进程控制

- `sys fork()`和`sys clone()`都通过调用`do_fork()`函数来完成进程的创建
- **`sys fork()`**: 除了代码段外, 父子进程不共享任何东西 (各自有一份)
- **`sys clone()`**: 有选择地对父进程进行复制, 可根据提供的`clone_flags`决定共享的内容
 - **不提供标志**: `sys clone()`与`sys fork()`相同
 - **提供标志**: 根据标志进行相应进程的共享, 就是传统的线程。如可共享页表、打开的文件列表、信号处理。因此, 利用`sys clone()`轻量级进程允许父子进程共享许多数据结构。

2.7 Linux中的进程控制

`sys vfork()`:

- 使用`vfork()`创建的新进程能够共享父进程的内存地址空间。使用复制指针的方法使子进程与父进程的资源实现共享。
- 父进程在这个过程中被阻塞, 直到子进程退出或者通过`exec()`执行一个新的程序租用父进程的地址空间。因此, `sys vfork()`保证子进程先运行, 在它调用`exec()`或`exit()`之后父进程才可能被调度运行 可以理解为: `vfork`创建新进程的目的在于`exec()`一个新进程。

`sys_vfork()`在写时复制技术出现之后, 就基本上不用它了

2.7 Linux中的进程控制

```
sys_fork()
|-do_fork()
|-copy_process()
|-dup_task_struct()
|-wake_up_new_task()
```

2.7 Linux中的进程控制

- ◆ 阅读 `do fork()`, 了解大致程序流程
- ◆ ??? 子进程从哪里开始执行, 它的返回值是什么?

2.7 Linux中的进程控制

- ◆ 大多情况下, 子进程从`fork()`返回后都调用`exec()`函数来执行新的程序
 - 进程调用`exec()`函数时, 该进程完全由新程序替代, 新程序从`main()`开始执行
 - `exec()`并不创建新进程, 前后进程ID不变, 但用另外一个程序替代当前进程的正文、数据、堆栈等

2.7 Linux中的进程控制

```
If (result = fork() == 0){
    /* 子进程代码 */
    ~
    if (execve("new_program",...) < 0)
        perror("execve failed");
    exit(1);
} else if (result < 0){
    perror("fork failed")
}
/* result == 子进程的pid, 父进程将会从这里继续执行*/
```

2.7 Linux中的进程控制

```
#include <unistd.h>

extern char **environ;

int exec( const char *path, const char *arg, ...);
int execl( const char *file, const char *arg, ...);
int execl( const char *path, const char *arg, ..., char * const envp[]);
int execv( const char *path, char * const argv[]);
int execvp( const char *file, char * const argv[]);
```

- ◆ 用一个指定的可执行文件所描述的上下文代替进程的上下文
- ◆ ➡ 系统调用: `execve()`
 - `_sys_execve()`

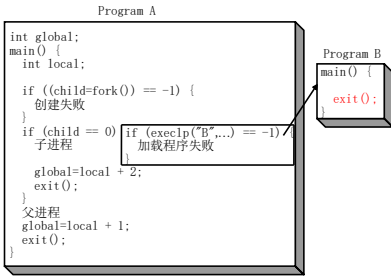
2.7 Linux中的进程控制

- 很多情况下, 子进程从`fork()`返回后很多会调用`exec()`来开始执行新的程序, 从而可以让父进程和子进程执行不同的任务。
- 因此, 子进程根本不需要读或者修改父进程拥有的所有资源。所以`fork()`中地址空间的复制依赖于Copy On Write技术, 以降低`fork()`的开销。
- 当进程调用`exec()`函数族时, 先找到并打开给定的可执行程序文件, 装入并运行目标程序。
- 如果`exec()`成功, 调用者进程将被覆盖, 从新进程的入口地址开始执行。

2.7 Linux中的进程控制

- `exec()`并不创建新进程, 前后进程ID是不变的。它是用另外一个程序替代了当前进程的正文、数据、堆和栈。
- `exec()`函数族的函数执行成功后不会返回, 因为调用进程的实体, 包括代码段、数据段和堆栈等都被新的内容取代, 只留下进程ID等一些表面上的信息仍保持原样

2.7 Linux中的进程控制



2.7 Linux中的进程控制

- ◆ 分开这两个系统调用是有好处的
 - 比如服务器可以fork()许多进程执行同一个程序
 - 有时程序只是简单的exec(), 执行一个新程序
 - 在fork和exec之间, 子进程可以有选择的执行一系列操作以确保程序以所希望的状态运行
 - 重定向输入输出
 - 关闭不需要的打开文件
 - 改变UID或是进程组
 - 重置信号处理程序
- ◆ 若单一的系统调用试图完成所有这些功能将是笨重、低效
 - 现有的fork-exec框架灵活性更强
 - 清晰, 模块化强

2.7 Linux中的进程控制

- ◆ 创建子进程后, 父进程的选择有以下几种:
 - (1) 父进程不理睬子进程, 继续执行。如果子进程先于父进程消亡的话, 则内核会发送一个信号, 通知父进程。
 - (2) 父进程暂停, 睡眠, 等待子进程结束后继续运行
 - (3) 父进程自行结束, 使用exit()。

2.7 Linux中的进程控制

- ◆ 父进程调用wait()类系统调用检查子进程是否终止
- ◆ 父进程调用wait()后进入阻塞队列中。当子进程结束时会产生一个终止状态字, 系统内核再向父进程发出SIGCHLD信号。当接收到信号时, 父进程提取子进程的终止状态字, 从wait()返回到原程序。
- ◆ Wait()系统调用的作用
 - 获取子进程终止的消息
 - 清除子进程的所有独占资源

2.7 Linux中的进程控制

- ◆ sys_wait4()算法的基本思路:
 - 搜索当前进程的所有子进程,
 - 如果父进程没有子进程, 则出错返回。
 - 如果找到一个满足要求的子进程, 但都不处于僵死状态, 则父进程或者睡眠等待, 以后由相应信号唤醒, 或者立即返回
 - 如果找到一个满足要求的子进程, 则返回子进程的进程号, 把子进程的CPU使用时间等加到父进程上, 释放子进程占用的task_struct和系统堆栈, 以供新进程使用(将其释放)。
- ◆ release()的唯一参数是要回收进程的task_struct结构, 与该结构相关的大部分资源已经在exit()中释放掉了。

2.7 Linux中的进程控制

- ◆ 函数原型
 - pid_t wait(int*status);
 - pid_t waitpid(pid_t pid, int*status, int options);
- ◆ 说明
 - 均通过wait4()系统调用实现
 - 进程终止时, 会向父进程发送SIGCHLD信号
- ◆ 调用wait()和waitpid()的进程的可能状态
 - 阻塞
 - 如果子进程还在运行
 - 正常返回
 - 返回子进程的终止状态(其中一个子进程终止)
 - 出错返回
 - 没有子进程

2.7 Linux中的进程控制

- ◆ 撤销时机
 - 主动撤销: 执行完代码, 通知内核释放进程的资源
 - 被动撤销: 内核强迫杀死进程
 - 进程接收到一个不能处理或忽视的信号
 - 在内核态产生一个不可恢复的CPU异常, 而内核此时正代表该进程在运行
- ◆ 撤销相关进程状态
 - 进程已死, 但必须保存它的描述符, 在得到通知后才可以删除
 - 僵死状态, 表明进程已死, 但需要等待父进程删除(如通过wait())

2.7 Linux中的进程控制

- ◆ 进程终止
 - 释放进程占有的大部分资源
 - 进程终止的一般方式是exit()或_exit()系统调用
 - 该系统调用可由编程者明确地在代码中插入
 - 控制流到达主过程最后一条语句时, 自动执行exit()
 - 信号机制
- ◆ 进程删除
 - 彻底删除进程的所有数据结构
 - 父进程调用wait()类系统调用获知子进程合法终止后, 即可删除该进程。

2.7 Linux中的进程控制

- ◆ 进程终止函数原型
 - exit(int status)
 - _exit(int status)
- ◆ status说明
 - 正常结束时返回, 否则表示出错信息
- ◆ 两函数共性
 - 都通过do_exit()系统调用
- ◆ 两函数区别
 - asmlinkage long sys_exit(int error_code)
 - {
 - do_exit((error_code&0xff)<<8);
 - }
 - _exit()直接使进程停止工作, 做清除和销毁工作
 - exit()在调用do_exit()系统调用前检查文件打开情况, 清理O缓冲

2.7 Linux中的进程控制

- 进程终止时要作的几件事
 - 通知父进程自己将要终止，通常利用信号通知机制，这可以通过`exit`函数实现
 - 父进程要有相应的机制响应进程的终止信号。通过信号处理程序和`wait4()`函数实现
 - 当父进程在子进程前终止时，要为其找一个父进程，如`init`进程
- 进程的终止分2步：
 - 自己调用函数`exit()`将自己的状态变为僵死状态，而后向父进程发送终止信号。
 - 由父进程调用`wait()`函数，回收僵死状态的子进程，将其从内存中彻底清除。

2.7 Linux中的进程控制

- 进程终止的一般方式是`exit()`系统调用
 - 这个系统调用可能由编程者明确的在代码中插入
 - 另外，在控制流到达主过程C中的`main()`函数的最后一条语句时，执行`exit()`系统调用
- 内核可以强迫进程终止
 - 当进程接收到一个不能处理或忽视的信号时
 - 当在内核态产生一个不可恢复的PU异常而内核此时正代表该进程在运行

2.7 Linux中的进程控制

- 进程终止使用`exit`系统调用

```
asm linkage long sys_exit(int error_code)
{
    do_exit((error_code&0xff)<<8);
}
```
- 删除进程
 - 在父进程调用`wait()`类系统调用检查子进程是否合法终止以后，就可以删除这个进程
 - 若子进程包含终止代号，则父进程通过`clearerr()`释放僵死进程的描述符
 - 释放进程d
 - 从进程链表中删除进程描述符
 - 释放存放进程描述符的内存区

2.7 Linux中的进程控制

- C编译器总是把`exit()`插入到`main()`的最后一条语句之后，`exit()`调用`_exit()`系统调用(kernel/exit.c中)
- `_exit()`调用`do_exit()`释放进程所占资源，终止进程
- 函数`do_exit()`释放进程占用的大部分资源
 - `do_exit()`调用`_exit_mm()`、`_exit_files()`、`_exit_fs()`以及`_exit_sighand()`
- 进程进入TASK_ZOMBIE状态(成为僵尸进程)，内核调用`exit_notify()`通知父进程料理后事。当父进程调用`wait()`函数族后，僵尸进程所占有的`task_struct`空间才被回收。
- 如果父亲先于子进程结束，则由父亲的父亲进程或者`init`进程来完成回收任务。

2.7 Linux中的进程控制

- `do_exit()`函数删除内核对终止进程的大部分引用：
 - 信号量队列中的进程描述符
 - 删除进程描述符中与分页、文件系统、打开文件描述符和信号处理相关数据结构
 - 减小进程所用模块的引用计数
- 将进程描述符的`exit_code`字段设置为终止代号
- 更新父子进程亲属关系，强制将自己的子进程作为其它某个进程的子进程，以等待该父进程来删除
- 调用`schedule()`进行调度

2.7 Linux中的进程控制

- 进程运行结束时要释放相应的资源，通过`exit()`调用实现(显式或隐式)
- `exit()`实现时调用了`do_exit()`完成以下工作
 - `Task_struct`中标志成员设为: `PF_EXITING`
 - 调用`_exit_mm()`
 - 调用`sem_exit()`
 - 调用`_exit_files()`、`_exit_fs()`、`exit_name_space`、`exit_sighand`
 - 退出代码替换为`EXIT()`提供的代码
 - 调用`Exit_notify()`向父进程发信号，(标为OOMBIE)
 - 调用`schedule`切换到其他进程

2.7 Linux中的进程控制

- 进程终止时，删除进程描述符
 - 父进程得知子进程终止的消息后才能删除子进程的描述符。(思考:若父进程异常终止,将会发生什么情况)
 - 父进程`WAIT()`, 返回时可以根据代码做相应动作(或者`IGNORE`), 动作完成后真正释放描述符(调用`release_task()`)
 - 调用`free_uid`:进程记数
 - `Unhash_process()`: 删除`pidhash`中的项同时删除`task_list`中的项
 - 若有`trace`删除`ptrace_list`中项
 - 调用`put_task_struct`释放描述符, (内核栈、`threadinfo`所占的页、`slab`高速缓存。

2.8 Linux线程实现机制

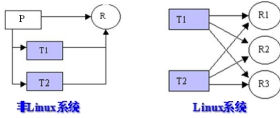
- Linux的线程描述机制
- Linux的线程分类
- Linux线程的创建

2.8 Linux线程实现机制

- 其他多线程操作系统
 - 通过两种数据结构来表示线程和进程: **线程和线程表项**
 - 每个进程可以指向若干个线程, 调度器在进程的时间片内再调度线程
- Linux操作系统
 - 从内核角度, 没有线程概念, 所有线程当做进程来实现
 - 线程被视为与其他进程共享某些资源的进程, 都有自己的进程描述符`task_struct`
 - 内核没有针对线程的调度算法或数据结构

2.8 Linux线程实现机制

- 举例（包含两个线程的进程）
 - 对于非Linux系统
 - 有一个包含两个不同线程指针的进程描述符
 - 每个线程指针描述其独占资源
 - 对于Linux系统
 - 创建两个进程并分配两个普通task_struct结构
 - 建立两个进程时只要指定它们共享的某些资源



2.8 Linux线程实现机制

- Linux内核中没有线程的概念
 - 没有针对所谓线程的调度策略
 - 没有数据结构用来表示一个线程
 - 一般线程的概念在linux中只是表现为一组共享资源的进程(每个这样的进程都有自己的进程描述符)
- 在其他系统中比如(windows)
 - 线程是实实在在的一种运行抽象，提供了比进程更轻更快的调度单元
 - 在Linux中“线程”仅仅是表示多个进程共享资源的一种说法

2.8 Linux线程实现机制

- 用户线程
 - 存在于用户空间中，通过线程库来实现
 - 线程创建和调度都在用户空间进行，以进程为单位调度
- 特点
 - 同一进程内的线程切换不需要转换到内核
 - 系统调度阻塞问题，不能充分利用多处理器
- 内核线程
 - 在内核空间内执行线程的创建、调度和管理，慢于用户线程的创建和管理
 - 支持多处理器，支持用户进程中的多线程、内核线程切换的速度快
 - 对用户的线程切换来说，系统开销大

2.8 Linux线程实现机制

- Linux线程的创建函数调用形式
 - `int clone(int (*fn)(void * arg), void *stack, int flags, void * arg);`
 - `int pthread_create(pthread_t *restrict tidp, const pthread_attr_t *restrict attr, void *(*start_rtn)(void), void *restrict arg);`
 - `tidp`: 新线程的线程描述表指针
 - `attr`: 为新线程定义不同属性如栈尺寸默认NULL
 - 第三个和第四个参数指定执行的函数`start_rtn`和传递给函数的参数`arg`
- 两者区别
 - `clone()`: 创建内核支持的用户线程，对内核可见且由内核调度
 - `pthread_create()`: 由基于POSIX标准的线程库创建的用户线程
 - 但在Linux里，`pthread_create()`最终调用`clone()`实现

2.8 Linux线程实现机制

- Linux内核线程独立运行在内核空间的标准进程，支持内核在后台执行一些操作
 - 刷新磁盘高速缓存
 - 交换出不用页面
 - 维护网络链接等待
- 与普通进程的区别
 - 只运行在内核态，内核线程没有独立的地址空间（`mm`指针被设置为NULL）
 - 每个内核线程执行一个单独的内核函数
 - 只使用大于`PAGE_OFFSET`的线性地址空间

2.8 Linux线程实现机制

- 没事闲逛的0号进程——从无到有诞生的第一个内核线程，执行`cpu_idle()`函数（省电又少热）。
- 既是内核线程也是用户进程的`init`。
- `init`内核线程被调度到时，执行内核函数`init()`，将会四次调用`kernel_thread`为常规内核任务初始化四个内核线程。
- `kflushd`（即`bdflush`）线程：刷新“脏”缓冲区中的内容到磁盘以归还内存。
- `Kupdatd`线程：刷新旧的“脏”缓冲区中的内容到磁盘以减少文件系统不一致的风险。
- `Kpiod`线程：把属于共享内存映射的页面交换出去。
- `Kswapd`线程：执行内存回收功能。
- `init`内核函数调用`execve()`，装入用户态下的可执行程序`sbin/init`，用户态下的第一个进程`init`就产生了。

本章知识点小结

- 理解和掌握Linux的进程/线程描述机制
 - Linux进程描述符
 - Linux进程的系统堆栈结构
- 理解和掌握Linux进程的内存管理机制
 - 内核链表结构
 - Linux的进程家族结构
 - Linux进程系统启动过程
- 理解和掌握Linux进程创建与实现机制
 - `fork()`、`vfork()`、`clone()`的使用场景及实现机制的差别
 - `do_fork()`的工作机理
- 理解和掌握Linux线程创建与实现机制
 - Linux线程的实现机制
 - Linux用户线程/内核线程的创建方法

作业

- Linux怎样描述进程？Linux有哪几种进程状态？画出Linux进程状态转换图。
- 查阅资料，在Linux2.6中，进程控制块如何存放？为什么？
- 0号进程来自哪里？在系统启动正常后，0号进程在什么情况下才被调度执行？1号进程的主要作用？
- 什么是僵尸进程？内核怎样处理僵尸进程？
- 创建进程的系统函数有哪几个？它们有什么区别？
- 简述进程终止的大致过程。
- Linux为什么要引入`pidhash`表？
- 在传统的UNIX系统中，创建子进程时会复制父进程的所有资源，代价比较高，现代UNIX系统中引入了哪几项技术来解决这个问题？
- 进程描述符放在动态内存中，而且和内核态的进程栈放在一个独立的KKB的内存区中。这样做有什么好处？